

UNIVERSIDAD AUTÓNOMA DE SANTO DOMINGO

CURSO-UASD (SANTO DOMINGO)

PRESENTACIÓN

Docente(s)

Radhames Silverio y Marlenis Concepción

Materia

Laboratorio de Lenguaje de Programación III (INF 5170)

Tema

Practica 6: XML / JSON (AJAX y su funcionamiento)

Sección

01

Nombre
Miguel Enmanuel Castillo Garcia

Matrícula
DA1734

Fecha

10/10/2026

AJAX y su funcionamiento

Introducción

Las aplicaciones web modernas necesitan comunicarse constantemente con servidores para consultar, enviar y actualizar información. Una de las técnicas que transformó la forma de realizar estas operaciones fue AJAX, acrónimo de Asynchronous JavaScript and XML. AJAX permite realizar solicitudes HTTP de manera asíncrona y utilizar la respuesta para actualizar partes de una página sin necesidad de cargar nuevamente el documento completo (MDN Web Docs, 2025).

Aunque el nombre AJAX hace referencia a XML, la técnica no depende de que los datos intercambiados sean XML. En la actualidad, JSON es un formato muy utilizado en este tipo de comunicación y la API Fetch se emplea con frecuencia para realizar solicitudes de red. MDN señala que Fetch es una alternativa más moderna y flexible a XMLHttpRequest, la API que originalmente se utilizó ampliamente para implementar AJAX (MDN Web Docs, 2025).

Este trabajo explica qué es AJAX, su propósito, su funcionamiento en una página web, la relación entre JavaScript, el navegador y el servidor, sus ventajas y desventajas, sus usos actuales y la diferencia entre una página tradicional y una aplicación que realiza solicitudes asíncronas. Finalmente, se presenta un ejemplo sencillo de funcionamiento.

1. ¿Qué es AJAX?

AJAX es una técnica de desarrollo web que permite que una aplicación realice solicitudes HTTP asíncronas desde JavaScript y utilice la respuesta para actualizar una parte de la interfaz sin requerir una recarga completa de la página. El término significa Asynchronous JavaScript and XML, aunque actualmente las respuestas pueden utilizar formatos como JSON, texto o HTML (MDN Web Docs, 2025).

Inicialmente, AJAX se implementaba principalmente mediante XMLHttpRequest. Esta API permite que una aplicación web realice solicitudes HTTP y reciba respuestas de un servidor mediante JavaScript, haciendo posible actualizar solamente la sección necesaria de la página (MDN Web Docs, 2025).

2. ¿Cuál es el propósito de AJAX?

El propósito de AJAX es mejorar la interacción entre el usuario y una aplicación web permitiendo que el navegador solicite o envíe información al servidor sin tener que reconstruir y cargar nuevamente toda la página.

Por ejemplo, cuando un usuario realiza una búsqueda, una aplicación puede enviar el texto introducido al servidor, recibir los resultados y actualizar solamente el área de resultados. De esta manera, el usuario puede continuar interactuando con el resto de la página mientras se realiza la comunicación con el servidor.

3. ¿Cómo funciona AJAX en una página web?

El funcionamiento puede explicarse mediante una secuencia de pasos:

3.1. El usuario realiza una acción, como presionar un botón, escribir en un campo de búsqueda o enviar un formulario.

3.2. JavaScript detecta la acción y prepara una solicitud HTTP.

3.3. El navegador envía la solicitud al servidor mediante una API de comunicación como XMLHttpRequest o Fetch.

3.4. El servidor procesa la solicitud y genera una respuesta.

3.5. La respuesta regresa al navegador, normalmente en JSON, texto o HTML.

3.6. JavaScript procesa la respuesta.

3.7. El código JavaScript actualiza solamente la parte de la interfaz que necesita cambiar.

Flujo simplificado:

Usuario → JavaScript → Navegador → Solicitud HTTP → Servidor → Respuesta → JavaScript → Actualización de la página

4. Relación entre AJAX, JavaScript, el navegador y el servidor

Componente	Función	Relación con AJAX
JavaScript	Ejecuta lógica en el navegador y responde a las acciones del usuario.	Inicia las solicitudes y procesa las respuestas.
Navegador	Ejecuta la página y proporciona APIs de comunicación de red.	Envía las solicitudes HTTP y recibe las respuestas.
Servidor	Procesa solicitudes y proporciona o modifica información.	Recibe la petición y devuelve los datos solicitados.
HTTP	Protocolo utilizado para la comunicación entre cliente y servidor.	Transporta las solicitudes y respuestas.

Por lo tanto, AJAX no es un lenguaje de programación ni una tecnología aislada. Es una técnica que combina JavaScript, las capacidades del navegador y la comunicación HTTP con un servidor. La API utilizada para la comunicación puede ser XMLHttpRequest o, en aplicaciones modernas, Fetch.

5. Ventajas y desventajas del uso de AJAX

5.1. Ventajas

- Permite actualizar partes de una página sin recargar el documento completo.
- Puede mejorar la sensación de rapidez y fluidez de una aplicación.
- Reduce la cantidad de contenido que necesita volver a cargarse cuando solo una sección cambia.
- Permite crear interfaces web interactivas y dinámicas.
- Facilita la comunicación con APIs y servicios web.
- Permite realizar consultas y enviar información mientras el usuario continúa trabajando con la interfaz.

5.2. Desventajas

- La aplicación depende de JavaScript para realizar y procesar las solicitudes.
- Errores de red o del servidor deben ser manejados correctamente para evitar una mala experiencia de usuario.
- Las políticas de seguridad del navegador, como CORS, pueden limitar solicitudes hacia otros orígenes.
- Una aplicación que realiza muchas solicitudes innecesarias puede aumentar el consumo de recursos y afectar el rendimiento.
- Una implementación deficiente puede complicar la accesibilidad, navegación y mantenimiento de la aplicación.

6. Usos actuales de AJAX

- **Formularios:** Enviar información al servidor y mostrar el resultado sin recargar toda la página.
- **Búsquedas:** Consultar resultados mientras el usuario interactúa con un campo de búsqueda.
- **Validaciones:** Consultar al servidor para verificar información antes de completar una operación.
- **Carga de datos:** Solicitar registros adicionales cuando son necesarios.
- **Aplicaciones web dinámicas:** Actualizar componentes específicos de la interfaz según las acciones del usuario.
- **APIs:** Consumir servicios que proporcionan información mediante JSON u otros formatos.
- **Aplicaciones de una sola página:** Actualizar el contenido de la interfaz mediante solicitudes de red sin realizar una navegación tradicional para cada cambio.

7. Diferencia entre una página web tradicional y una página que utiliza AJAX

Aspecto	Página tradicional	Página con AJAX
Solicitud	La acción normalmente genera una navegación o recarga del documento.	Puede realizarse en segundo plano.
Actualización	Se vuelve a cargar la página completa.	Se actualiza solamente la sección necesaria.
Interacción	Puede interrumpirse mientras se procesa una navegación.	Puede continuar mientras la solicitud asíncrona se procesa.
Datos	Puede devolver una página HTML completa.	Puede devolver JSON, texto, HTML u otros datos.
Experiencia	Más dependiente de recargas completas.	Más dinámica e interactiva.

8. Ejemplo sencillo de funcionamiento de AJAX

El siguiente ejemplo utiliza la API Fetch para solicitar un archivo JSON y mostrar sus datos en una sección de la página. Fetch es la alternativa moderna a XMLHttpRequest para realizar solicitudes de red y utiliza Promises para manejar las respuestas (MDN Web Docs, 2025).

8.1. HTML

```
<!DOCTYPE html>
<html lang="es">
<head>
   <meta charset="UTF-8">
   <title>Ejemplo AJAX</title>
</head>
<body>

   <h1>Producto</h1>

   <!-- Botón que inicia la solicitud -->
   <button id="btnCargar">Cargar producto</button>

   <!-- Lugar donde se mostrará la respuesta -->
   <div id="resultado"></div>

   <script src="app.js"></script>
</body>
</html>
```

8.2. Archivo JSON

```
{
   "nombre": "Laptop Lenovo",
   "precio": 45000,
   "disponible": true
}
```

8.3.JavaScript

```
// Obtenemos el botón y el elemento donde mostraremos los datos.
const boton = document.getElementById("btnCargar");
const resultado = document.getElementById("resultado");

// Ejecutamos la solicitud cuando el usuario presiona el botón.
boton.addEventListener("click", () => {

   // Fetch realiza una solicitud HTTP de forma asíncrona.
   fetch("producto.json")

      // Convertimos la respuesta HTTP a un objeto JavaScript.
      .then(response => response.json())

      // Utilizamos los datos recibidos.
      .then(producto => {

         // Actualizamos solamente esta sección de la página.
         resultado.innerHTML = `
            <p>Nombre: ${producto.nombre}</p>
            <p>Precio: RD$ ${producto.precio}</p>
            <p>Disponible: ${producto.disponible ? "Sí" : "No"}</p>
         `;
      })

      // Manejamos posibles errores de comunicación.
      .catch(error => {
         resultado.textContent = "No fue posible cargar el producto.";
         console.error(error);
      });
});
```

8.4.Explicación del funcionamiento

1. El usuario presiona el botón Cargar producto.
2. El evento click ejecuta la función asociada.
3. fetch("producto.json") solicita el archivo al servidor.
4. La respuesta se convierte de JSON a un objeto JavaScript mediante response.json().
5. Los datos se utilizan para construir el contenido que se mostrará.
6. innerHTML actualiza solamente el elemento resultado.
7. La página completa no necesita recargarse para mostrar la información.

8.5. Ejemplo utilizando XMLHttpRequest

Para comprender el origen de la técnica AJAX, también es útil observar el mecanismo tradicional basado en XMLHttpRequest. Esta API permite realizar solicitudes HTTP y recibir respuestas programáticamente sin una recarga completa de la página (MDN Web Docs, 2025).

```
const xhr = new XMLHttpRequest();

// Preparar una solicitud GET asíncrona.
xhr.open("GET", "producto.json", true);

// Procesar la respuesta cuando la solicitud finalice.
xhr.onload = () => {
   if (xhr.status === 200) {
      const producto = JSON.parse(xhr.responseText);
      console.log(producto.nombre);
   }
};

// Manejar errores de red.
xhr.onerror = () => {
   console.error("Error al realizar la solicitud.");
};

// Enviar la solicitud.
xhr.send();
```

En este segundo ejemplo, XMLHttpRequest cumple la función que históricamente se utilizó para implementar AJAX. La API permite preparar la solicitud con open(), enviarla con send() y procesar la respuesta mediante eventos. Actualmente, Fetch suele ser la opción recomendada para nuevas aplicaciones porque ofrece una API más moderna y flexible (MDN Web Docs, 2025).

Conclusión

AJAX representa una técnica fundamental en la evolución de las aplicaciones web porque permite realizar comunicaciones asíncronas entre el navegador y el servidor y actualizar solamente las partes necesarias de una interfaz. Esto contribuye a crear experiencias más dinámicas e interactivas.

Su funcionamiento depende de la interacción entre JavaScript, el navegador, HTTP y el servidor. Históricamente, XMLHttpRequest fue la API principal para implementar este tipo de comunicación, mientras que Fetch se ha convertido en una alternativa moderna y flexible. Además, aunque el nombre AJAX contiene la palabra XML, actualmente es común intercambiar datos utilizando JSON.

Comprender AJAX resulta importante para un estudiante de desarrollo web porque permite entender cómo una aplicación puede comunicarse con servicios y APIs sin depender de recargas completas de las páginas. El ejemplo presentado demuestra este principio mediante una solicitud de datos y la actualización de una sección específica de la interfaz.

Referencias

- MDN Web Docs. (2025, 11 de julio). *Ajax*. https://developer.mozilla.org/en-US/docs/Glossary/AJAX

- MDN Web Docs. (2025). *XMLHttpRequest API*. https://developer.mozilla.org/en-US/docs/Web/API/XMLHttpRequest_API

- MDN Web Docs. (2025). *Using XMLHttpRequest*. https://developer.mozilla.org/en-US/docs/Web/API/XMLHttpRequest_API/Using_XMLHttpRequest

- MDN Web Docs. (2025). *Fetch API*. https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

- MDN Web Docs. (2025). *Making network requests with JavaScript*. https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Scripting/Network_requests